

Companion to Bird & de Moor, Algebra of Programming

§10. Greedy Algorithms

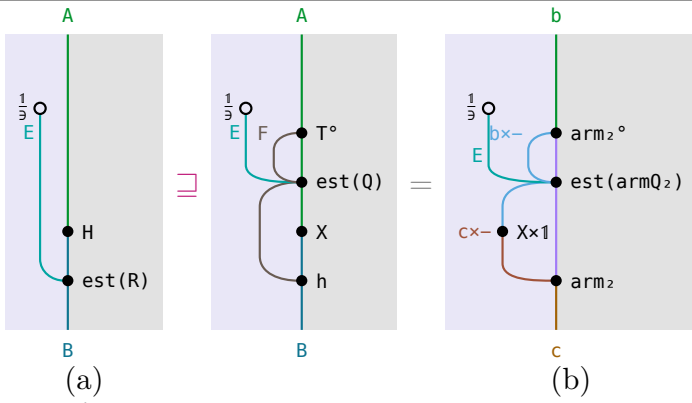
§10.1. Theory

(10.1a)

h, T, R, H, M as in $\triangle H$; additionally Q a **connected** preorder on the sets $\frac{T^\circ}{\exists}$ returns, so that $\frac{T^\circ}{\exists} \text{est}(Q)$ is entire.

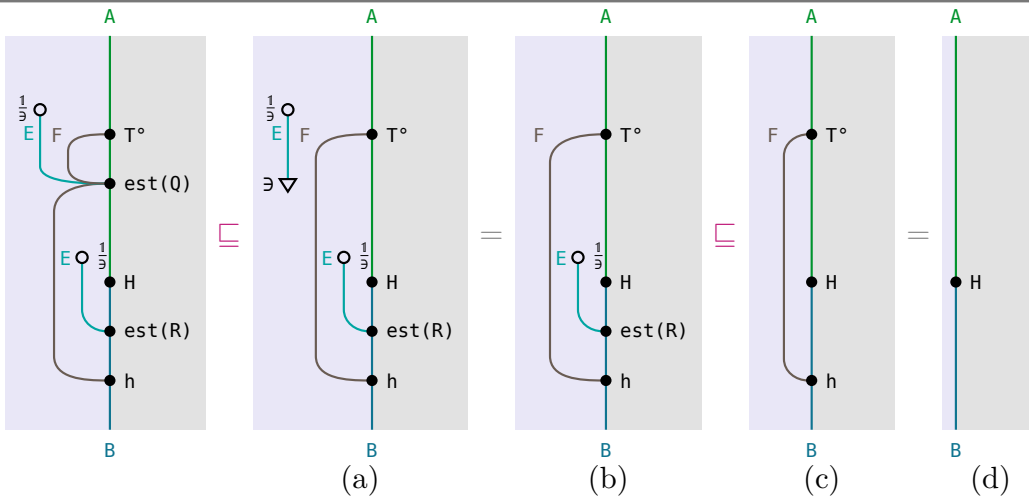
Map(h) $\wedge$ MonoAlg(h,R) $\wedge$ RR $\subseteq$ R $\wedge$ ThinCondition(T,h,R,Q) $\Rightarrow$ ($\mu X : \frac{T^\circ}{\exists} \text{est}(Q)F(X)h$) $\subseteq \frac{H}{\exists}$ **est(R)** Theorem 10.1

the same optimum reached by keeping ONE decomposition at each step, so that no set is ever carried and the recursion runs on values alone



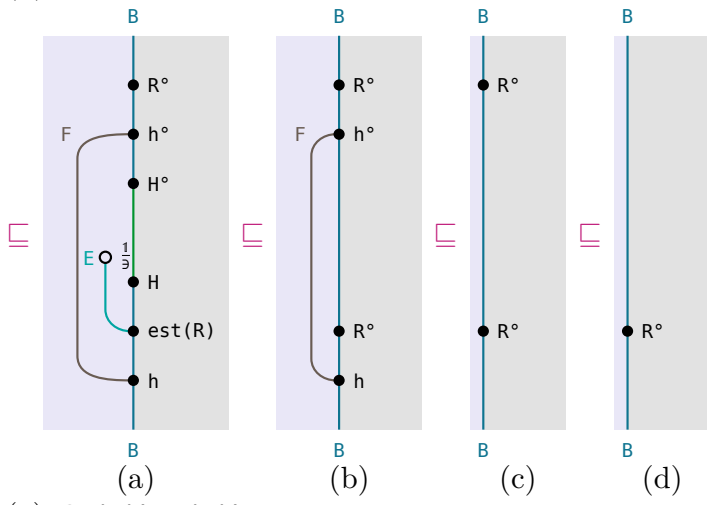
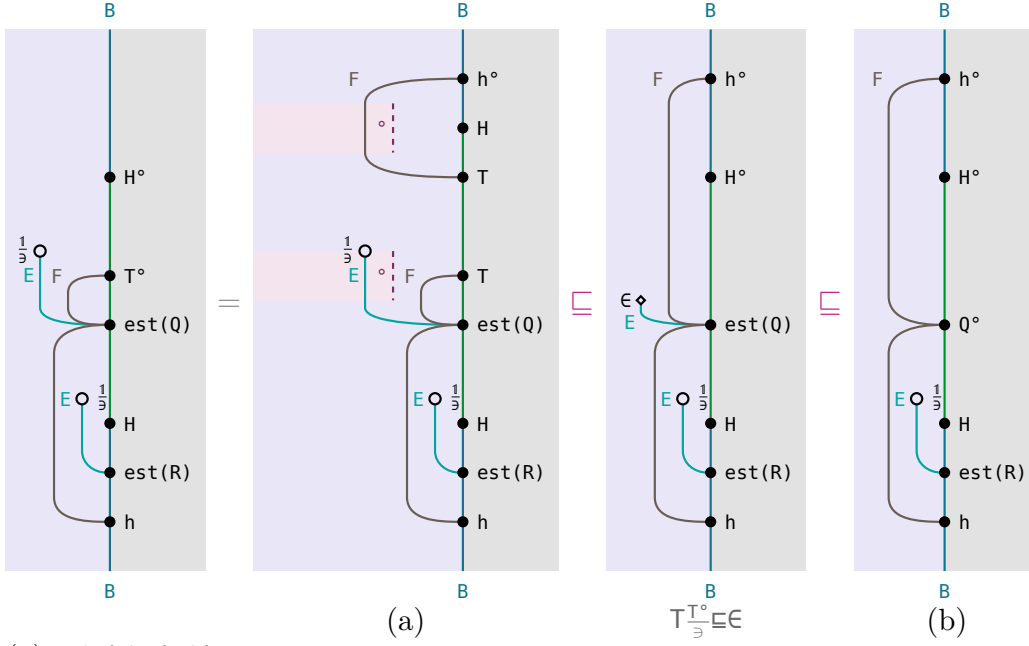
$T^\circ F(H)h=H \Rightarrow \frac{T^\circ}{\exists} \text{est}(Q)F(\frac{H}{\exists} \text{est(R)})h \subseteq H$ (10.1b)

taking the input apart in one Q -extreme way, solving the parts by M and assembling by h returns only what H returns



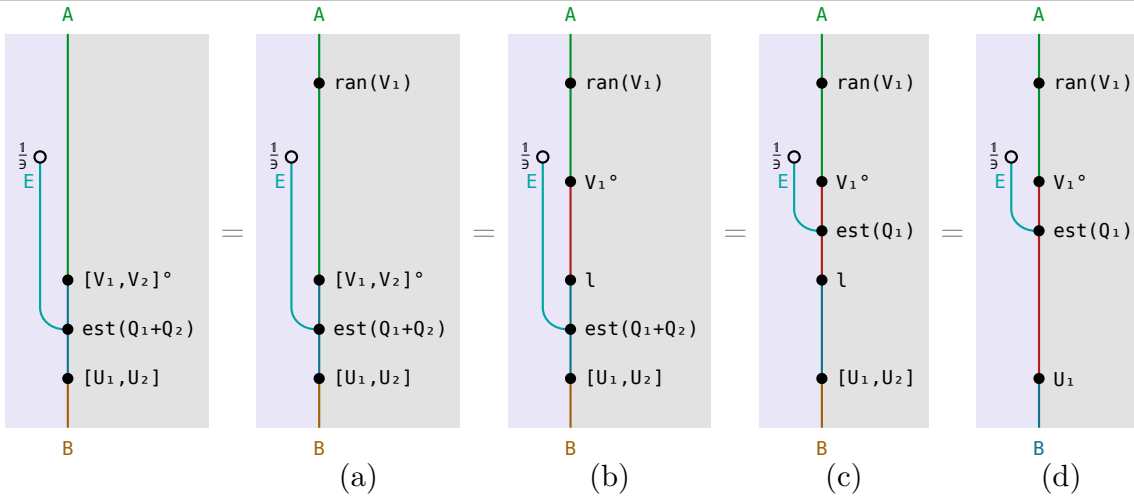
$$\text{Map}(h) \wedge \text{MonoAlg}(h, R^\circ) \wedge R^\circ R^\circ \sqsubseteq R^\circ \wedge T^\circ F(H)h = H \wedge QF(H)h \sqsubseteq F(H)hR \Rightarrow H^\circ \frac{T^\circ}{\exists} \text{est}(Q)F(\frac{H}{\exists} \text{est}(R))h \sqsubseteq R^\circ \quad (10.1c)$$

H° followed by the greedy body at M is $\sqsubseteq R^\circ$: an answer of the body is never worse than an answer of H to the same input



$$V_2 \quad V_1^\circ = \emptyset \Rightarrow \frac{[V_1, V_2]^\circ}{\exists} \text{est}(Q_1 + Q_2)[U_1, U_2] = \text{ran}(V_1) \frac{V_1^\circ}{\exists} \text{est}(Q_1)U_1 \cup \text{ran}(V_2) \frac{V_2^\circ}{\exists} \text{est}(Q_2)U_2 \quad \text{Proposition 10.1}$$

when V_1 and V_2 have disjoint ranges, the greedy step over $[V_1, V_2]$ runs the V_1 step on inputs V_1 reaches and the V_2 step on inputs V_2 reaches



§10.2. The detab-entab problem

(10.2a)

$\text{detab} \triangleq ([\text{nil}, \text{expand}]) : \text{String} \rightarrow \text{String}$ over snoc -lists, $\alpha \triangleq [\text{nil}, \text{snoc}]$, $H = \text{detab}^\circ$; $\text{expand } (xs, a) = (a = \text{TB} \rightarrow \text{fill } xs, xs \# [a])$, $\text{fill } xs = xs \# \text{blanks } (n - (\text{col } xs) \bmod n)$.

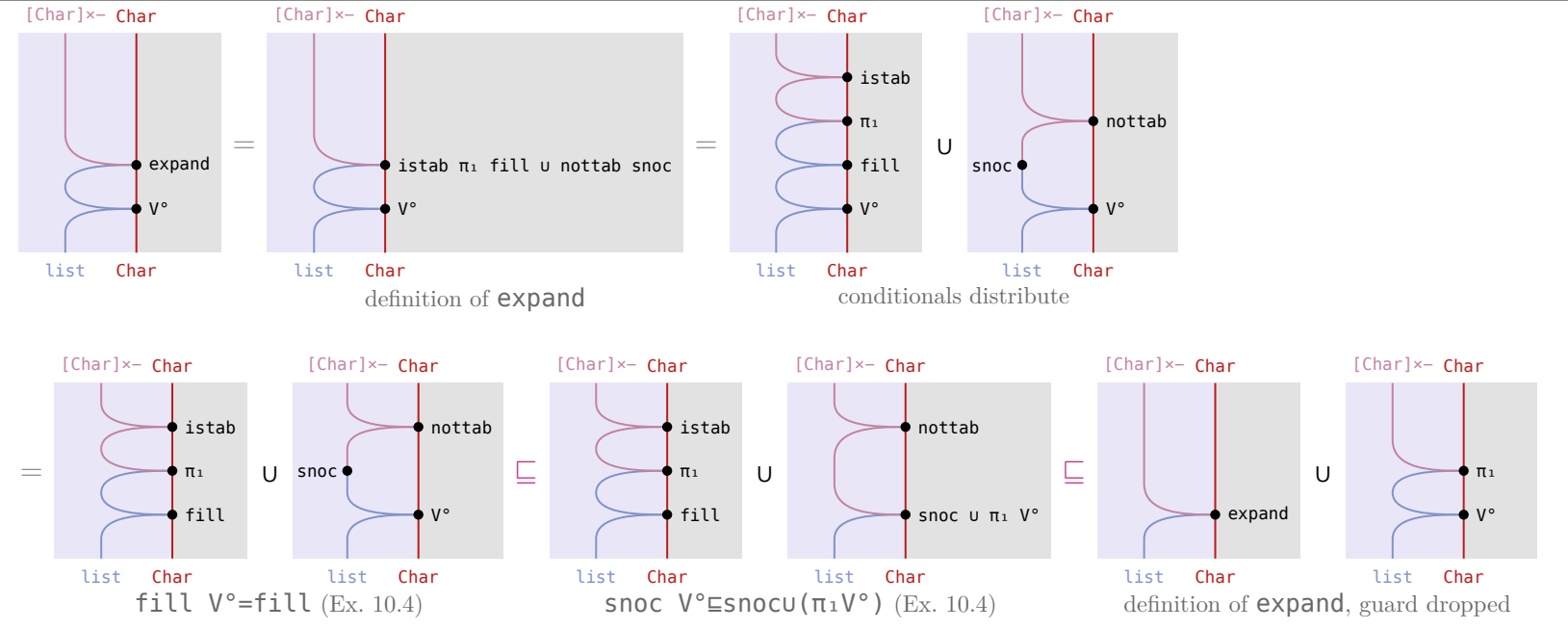
$\text{col} \triangleq ([\text{zero}, \text{count}])$, $\text{count } (c, a) = (a = \text{NL} \rightarrow 0, c + 1)$; TB the tab, BL the blank, NL the newline, tab stops every n columns.

$R \triangleq \text{length} \leq \text{length}^\circ$, U the preorder with $a \ U \ b \iff a = \text{TB} \vee a = b$, $V \triangleq \text{prefix}^\circ n (\text{fill } \text{fill}^\circ)$, $Q \triangleq 1 + (V \times U)$.

$\text{unfill } xs$ the shortest prefix of xs with fill ($\text{unfill } xs = \text{fill } xs$); tbc the trailing blank count, $\text{triple} \triangleq (\text{unfill } \text{entab}, (\text{tbc}, \text{col}))$.

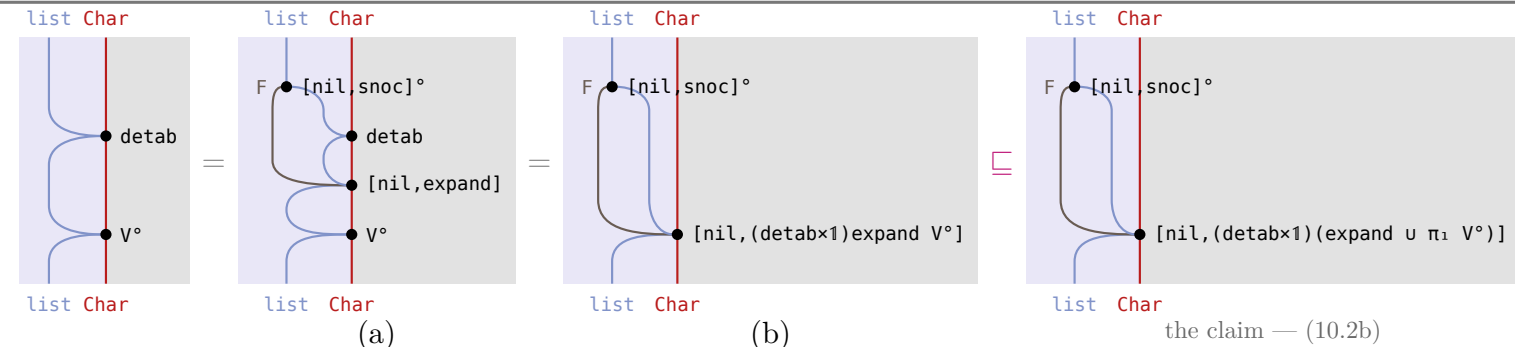
$$0 < n \wedge \text{blank} \neq \text{nl} \implies \text{expand } V^\circ \subseteq \text{expand} \cup \pi_1 V^\circ \quad (10.2b)$$

shortening the output of one **expand** step to a V -smaller string either keeps the whole step (**expand**) or drops its character and shortens its input string ($\pi_1 V^\circ$)



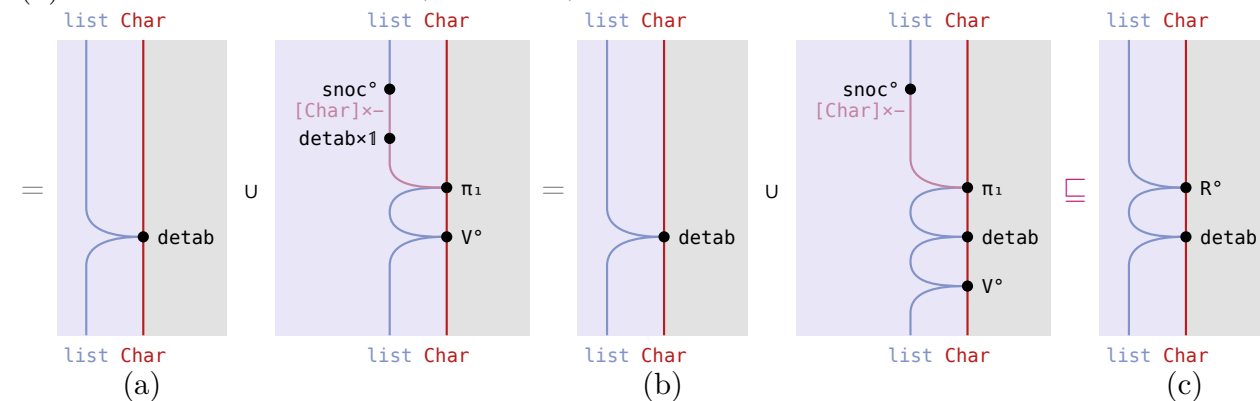
$$0 < n \wedge \text{blank} \neq \text{nl} \implies \text{detab } V^\circ \subseteq R^\circ \text{detab} \quad (10.2c)$$

any V -shortening of **detab**'s output is **detab**'s output on an input no longer than the given one



(a) **detab** is a fold: $\text{detab} = \alpha^\circ F(\text{detab}) [\text{nil}, \text{expand}] \triangleq \text{detab}$

(b) coproducts, and $\text{nil } V^\circ = \text{nil}$ (Exercise 10.4)



(a) distributing U ; the fold again, and the definition of F

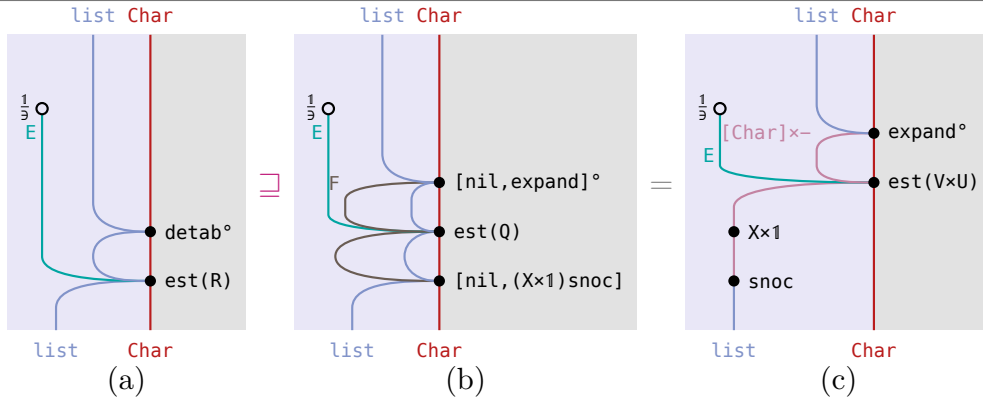
(b) naturality of π_1 : $(\text{detab} \times 1) \pi_1 = \pi_1 \text{detab}$; $\text{snoc}^\circ \pi_1$ is **init**

(c) **init** is inductive, so $X \triangleq \text{detab } V^\circ$, a solution of $X \subseteq \text{detab} \cup (\text{init } X)$, lies below the greatest one, **prefix detab** (induction on the input); $\text{prefix} \in R^\circ$

(10.2a)

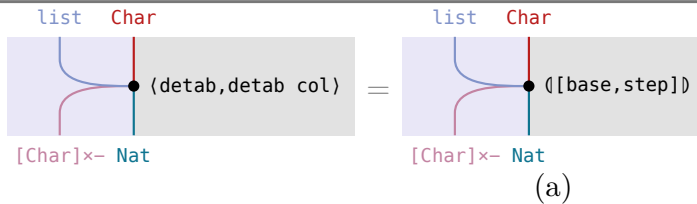
$$0 < n \wedge \text{blank} \neq n1 \Rightarrow (\mu X : \frac{[\text{nil}, \text{expand}]^\circ}{3} \text{est}(Q)F(X)[\text{nil}, \text{snoc}]) \sqsubseteq \frac{\text{detab}^\circ}{3} \text{est}(R) \quad (10.2d)$$

the shortest input **detab** expands to the given output is one pass along that output, holding each blank back and cashing the held blanks in for a tab wherever the column reaches a tab stop



$$\text{blank} \neq n1 \wedge \text{tb} \neq n1 \Rightarrow \langle \text{detab}, \text{detab col} \rangle = \langle [base, step] \rangle \quad (10.2e)$$

one pass over the input carries the output so far together with its column



Exercise 10.1

$$\text{blank} \neq n1 \wedge \text{tb} \neq n1 \Rightarrow \langle [base, step] \rangle (\text{convert}(l), r) \Leftrightarrow r = \text{loop}(\text{step}, (([], 0), l))$$

the fold of $[base, step]$ over the input converted to a snoc-list is the left loop of **step** from **base**

$$\pi_1(\text{loop}(\text{step}, (([], c_0), x))) = \text{loop}'((f, g), (c_0, x))$$

Exercise 10.1: when **step** only appends $f(c, a)$ to the output and moves the state by g , the output of the loop is $\text{loop}'(f, g)$, which never carries the output

$$a \neq \text{tb} \Rightarrow \text{expand}(y, b) = \text{snoc}(x, a) \Leftrightarrow \text{fill}(y) = \text{snoc}(x, a) \wedge b = \text{tb} \vee (y, b) = (x, a) \quad (10.2f)$$

expand produces $x \# [a]$ from a tab after a string that fills to it, or from x and a

$$0 < n \wedge \text{blank} \neq n1 \Rightarrow (\exists y. \text{fill}(y) = \text{snoc}(x, a)) \Leftrightarrow a = \text{blank} \wedge \text{col}(\text{snoc } x \text{ } a) \% n = 0$$

a string is a **fill** exactly when it ends in a blank on a tab stop

$$\text{entab}(x) = \text{entab}(\text{unfill } x) ++ \text{blanks}(\text{tbc } x) \quad (10.1)$$

(10.1): the output of **entab** is that of **unfill** x followed by the trailing blanks held back

$$\text{tbc}[] = 0$$

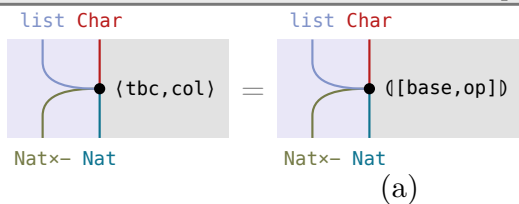
the empty string has no trailing blanks

$$\text{tbc}(\text{snoc}(x, a)) = \text{if } a = \text{blank} \wedge \text{col}(\text{snoc } x \text{ } a) \% n \neq 0 \text{ then } \text{tbc } x + 1 \text{ else } 0$$

a blank off a tab stop adds one held blank; anything else releases them all

$$\text{blank} \neq n1 \Rightarrow \langle \text{tbc}, \text{col} \rangle = \langle [base, op] \rangle \quad (10.2g)$$

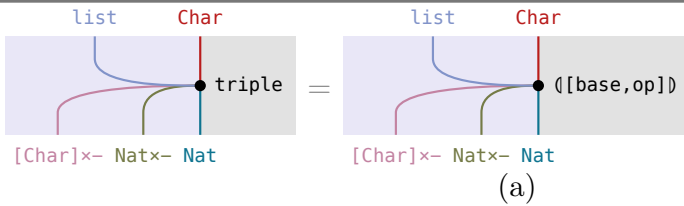
one pass counts the held blanks and the column together



$$\text{blank} \neq \text{nl} \Rightarrow \text{triple} = ([\text{base}, \text{op}])$$

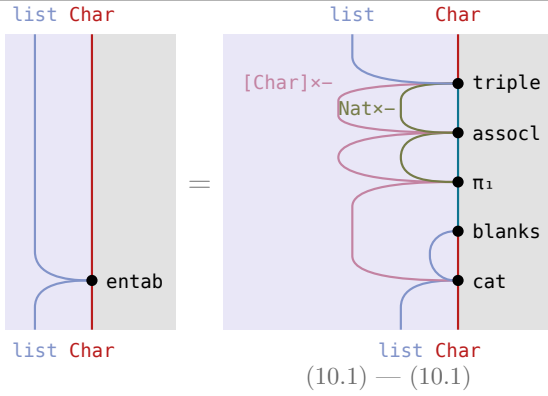
(10.2h)

one pass carries the output up to the held blanks, their count and the column

(a) op on $\text{triple } x$ is triple one character further: (10.1) where the blanks are released — (10.1)

$$\text{entab} = \text{triple} \text{ assocl } \pi_1 (1 \times \text{blanks}) \text{ cat}$$

(10.2i)

 entab is triple with the held blanks appended to its output

§10.3. The minimum tardiness problem

(10.3a)

$\text{FX} = 1 + (\text{X} \times \text{Job})$, $\alpha \triangleq [\text{nil}, \text{snoc}]$ on schedules, $\beta \triangleq [\text{nil}, \text{snag}]$ on bags, snag putting a job into a bag; $\text{bagify} \triangleq ([\beta]) : [\text{Job}] \rightarrow \text{Bag Job}$, $\text{H} = \text{bagify}^\circ$.

$\text{ct}, \text{dt}, \text{wt} : \text{Job} \rightarrow \text{Real}$ the completion, due and weighting quantities of a job; $\text{penalty}(\text{xs}, j) = (\text{sum}(\text{list}(\text{ct})(\text{xs})) + \text{ct}(j) - \text{dt}(j)) \times \text{wt}(j)$.

$\text{cost} \triangleq \frac{\text{prefix}}{\sup} P(\alpha^\circ [\text{zero}, \text{penalty}]) \text{ est}(\geq)$, $\text{cost} [] = 0$, $\text{cost} (\text{xs} \# [j]) = \text{bmax} (\text{cost } \text{xs}, \text{penalty} (\text{xs}, j))$, $\text{R} \triangleq \text{cost} \leq \text{cost}^\circ$.

$\text{perm} \triangleq \text{bagify } \text{bagify}^\circ = ([\text{nil}, \text{add}])$, $\text{add} (\text{xs}, j) = \text{ys} \# [j] \# \text{zs}$ for some $\text{xs} = \text{ys} \# \text{zs}$.

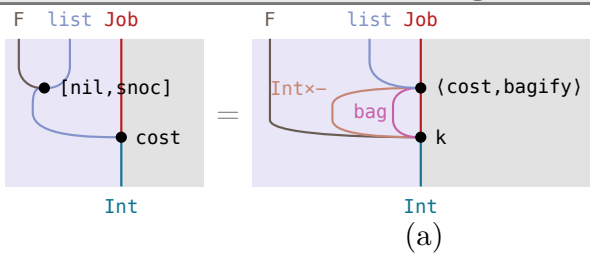
$\text{k} \triangleq [\text{zero}, \text{assocr} (1 \times ((\text{bagify}^\circ \times 1) \text{ penalty})) \text{ bmax}]$, $\text{f} \triangleq [\text{zero}, (\text{bagify}^\circ \times 1) \text{ penalty}]$, $\text{Q} \triangleq \text{f} \leq \text{f}^\circ$, $\text{Q}' \triangleq (\text{bagify}^\circ \times 1) \text{ penalty} \leq \text{penalty}^\circ (\text{bagify} \times 1)$.

$\text{g} \triangleq [\text{zero}, \text{penalty}]$ (10.5), $\text{m} \triangleq [\text{zero}, \pi_1 \text{ cost}]$ (B&dM's h , renamed as in (10.3e)) (10.6).

$$[\text{nil}, \text{snoc}] \text{ cost} = \text{F}((\text{cost}, \text{bagify})) \text{ k}$$

(10.3b)

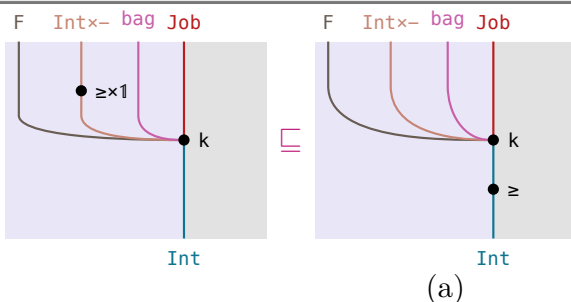
the cost of a schedule is got from the cost and the bag of the schedule before its last job, and that job

(a) $\text{cost} (\text{xs} \# [j]) = \text{bmax} (\text{cost } \text{xs}, \text{penalty} (\text{xs}, j))$, and $\text{penalty}(\text{xs}, j)$ reads only the bag of xs — $\triangleq \text{bagify}$

$$\text{F}(\geq \times 1) \text{ k} \leq \text{k} \geq$$

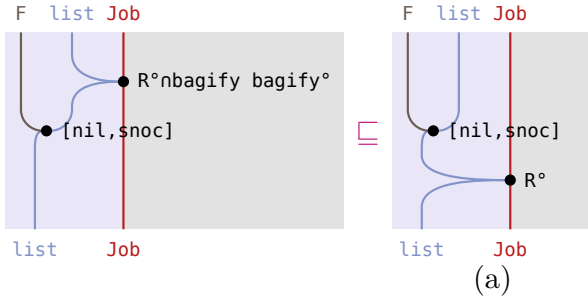
Exercise 10.5

a larger cost before the last job gives a larger cost after it

(a) bmax is monotone in its first argument

$$F(R^\circ \text{nbagify bagify}^\circ)[\text{nil}, \text{snoc}] \sqsubseteq [\text{nil}, \text{snoc}]R^\circ \quad (10.2)$$

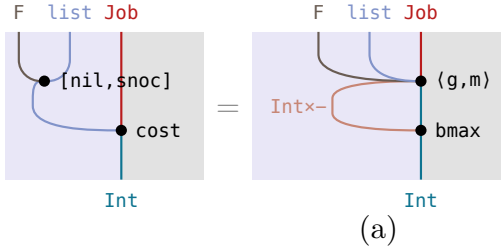
improving the schedule before the last job, among schedules of the same bag, improves the whole schedule



(a) Proposition 9.3 at $S \triangleq \text{bagify}$, $\leq \triangleq \text{dp-context-mono}$, with (10.3b) and Exercise 10.5

$$[\text{nil}, \text{snoc}] \text{cost} = (g, m) \text{bmax} \quad (10.4)$$

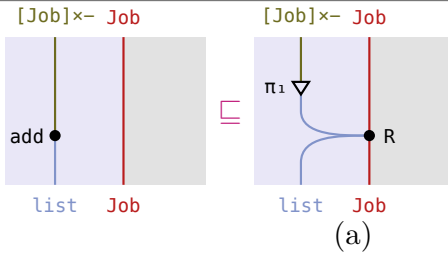
the cost of a schedule is the larger of the penalty of its last job and the cost of the schedule before it



(a) $\text{cost}(xs \# [j]) = \text{bmax}(\text{cost } xs, \text{penalty}(xs, j))$ and bmax commutes — $\triangleq \text{bagify}$

$$(\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow \text{add} \sqsubseteq \pi_1 R \quad (10.7)$$

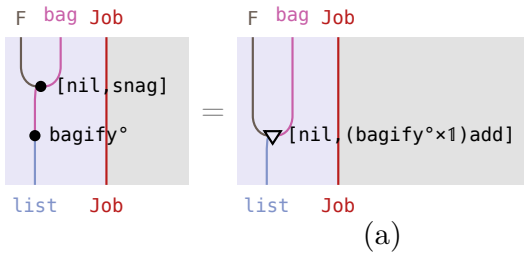
putting one more job anywhere into a schedule never lowers its cost



(a) induction on where j goes: every job after it starts later, so no penalty falls

$$[\text{nil}, \text{snag}] \text{bagify}^\circ = [\text{nil}, (\text{bagify}^\circ \times 1) \text{add}] \quad (10.8)$$

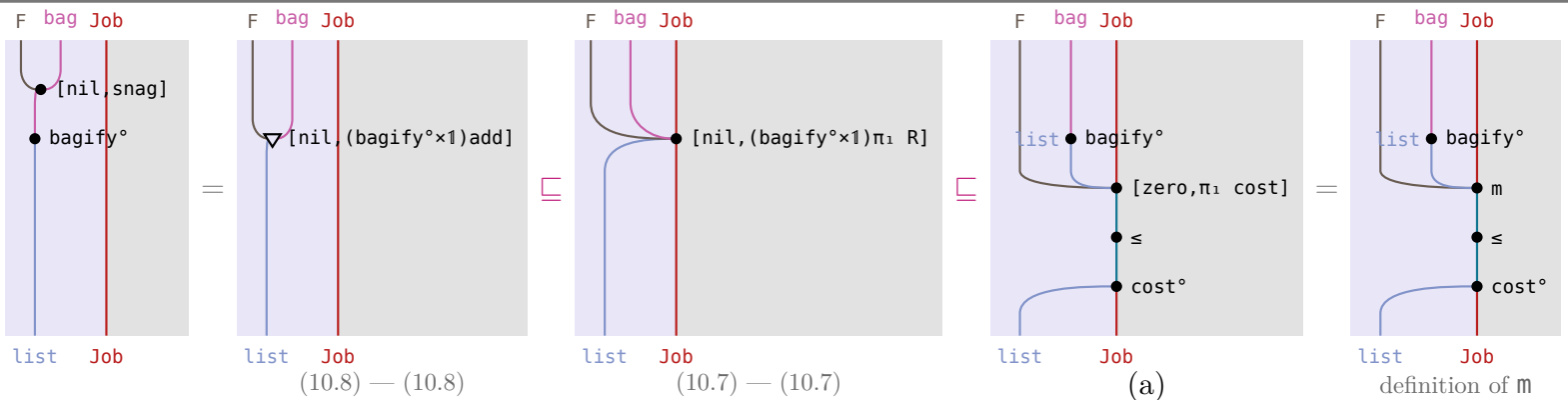
the orderings of a bag with one more job are the orderings of the bag with that job put in anywhere



(a) take out the last copy of j and put it back — $\triangleq \text{bagify}$

$$(\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow [\text{nil}, \text{snag}] \text{bagify}^\circ \sqsubseteq F(\text{bagify}^\circ) m \leq \text{cost}^\circ \quad (10.3c)$$

an ordering of a bag with one more job costs at least the cost of the ordering of the bag before its last job

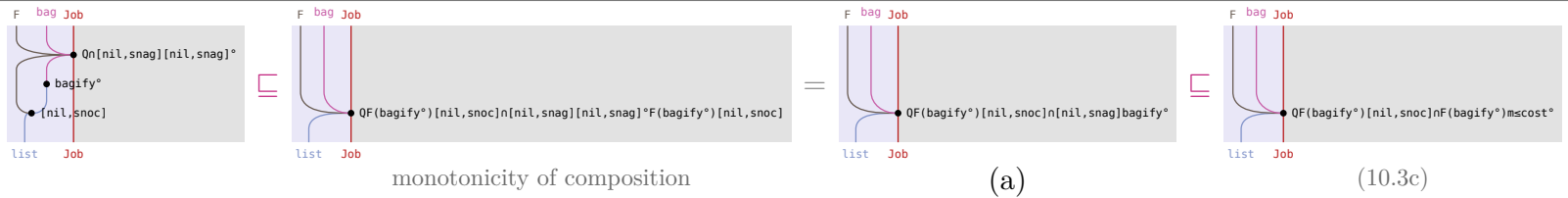


(a) definition of R , and $\text{nil} \sqsubseteq \text{zero} \leq \text{cost}^\circ$

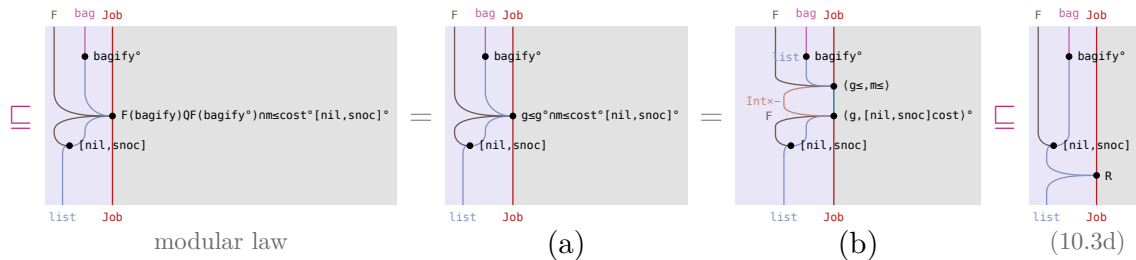
$$(\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow (Qn[\text{nil}, \text{snag}][\text{nil}, \text{snag}]^\circ F(\text{bagify}^\circ)[\text{nil}, \text{snoc}] \sqsubseteq F(\text{bagify}^\circ) [\text{nil}, \text{snoc}]R) \quad (10.3)$$

$[\text{nil}, \text{snoc}]R$

a schedule of a bag ending in a job of least penalty is no worse than any schedule of the same bag



(a) $\beta^\circ F(\text{bagify}^\circ)\alpha = \text{bagify}^\circ$, since $\text{bagify} = (\beta)$

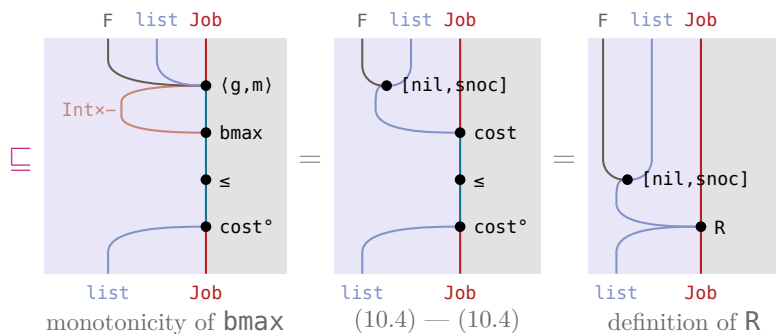
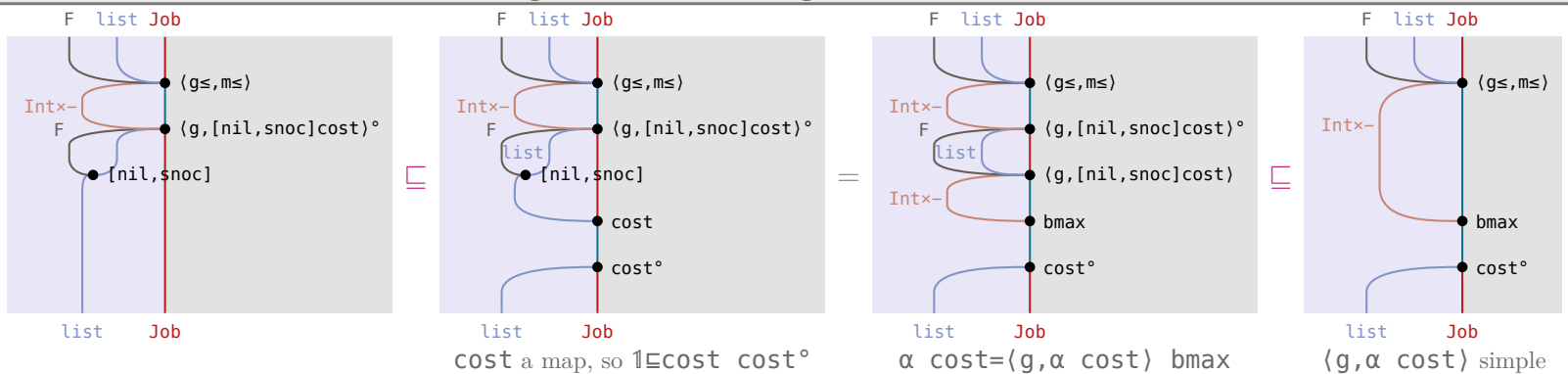


(a) choice of Q : $F(\text{bagify}) \ Q \ F(\text{bagify}^\circ) = g \leq g^\circ$

(b) products: $\langle R, S \rangle \langle T, U \rangle^\circ = RT^\circ \cap SU^\circ$

$$\langle g \leq, m \leq \rangle \langle g, [\text{nil}, \text{snoc}] \text{cost} \rangle^\circ [\text{nil}, \text{snoc}] \sqsubseteq [\text{nil}, \text{snoc}]R \quad (10.3d)$$

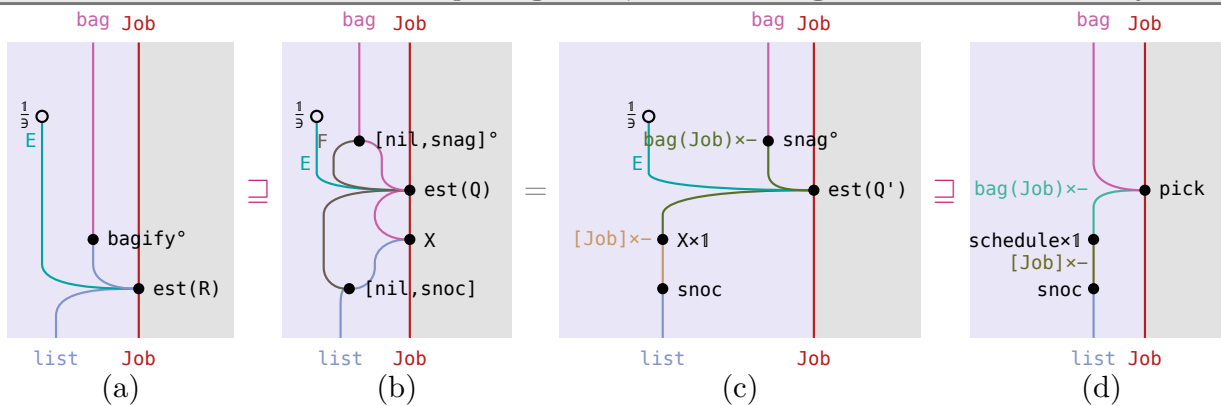
a job whose penalty is at most the last penalty, put after a schedule costing at most the cost before the last job, gives a schedule costing at most the whole



$$(\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow (\mu X : \frac{[\text{nil}, \text{snag}]^\circ}{3} \text{est}(Q)F(X)[\text{nil}, \text{snoc}] \sqsubseteq \frac{\text{bagify}^\circ}{3} \text{est}(R), \quad (10.3e)$$

schedule = (null-nil, pick (schedule×1) snoc)

an ordering of the given bag with least maximum penalty is got by taking a job of least penalty out of the bag, putting it last, and scheduling what is left the same way



(a) the specification — $\triangleq \text{bagify}$

(b) No greedy **reduce** exists — one would also solve every prefix of the input, and the best schedule of a prefix need not extend to a best schedule of the whole

(c) Proposition 10.1: nil and snag have disjoint ranges

(d) $\text{pick} \sqsubseteq \frac{\text{snag}^\circ}{3} \text{est}(Q')$, a partial function, quadratic in the number of jobs

§10.4. The TeX problem

(10.4a)

$\text{interval} \triangleq \text{val round} : \text{Decimal} \rightarrow [0, 2^{16})$, $\text{val} \triangleq ([\text{zero}, \text{shift}])$, $\text{shift } (d, r) = (d+r)/10$, $\text{round } r$ rounds $2^{16}r$ to the nearest integer: $\text{round } r = n \iff 2n-1 < 2^{17}r < 2n+1$.

$\text{interval } n = ((2n-1)/2^{17}, (2n+1)/2^{17})$, $r \text{ inrange } (a, b) \iff a < r < b$, $\text{round}^\circ = \text{interval inrange}$, $R \triangleq \text{length} \leq \text{length}^\circ$.

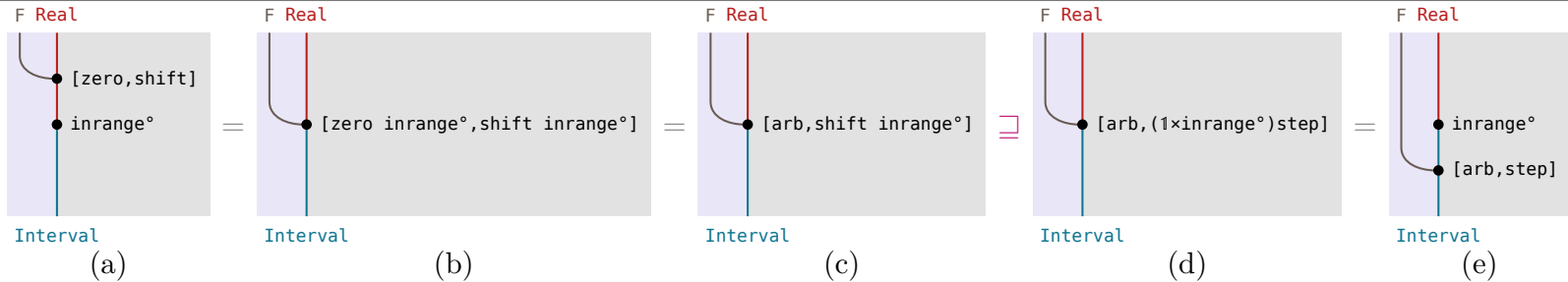
Interval the pairs (a, b) with $\text{Legal}(a, b) \iff 0 < b < 1$ and $a < b$ (10.9); step_legal is what types $\text{step } [\text{arb}, \text{step}] : 1 + (\text{Digit} \times \text{Interval}) \rightarrow \text{Interval}$, $\text{step } (d, (a, b)) = ((d+a)/10, (d+b)/10)$.

$\text{FX} = 1 + (\text{Digit} \times X)$, $\alpha \triangleq [\text{nil}, \text{cons}]$, $H \triangleq ([\text{arb}, \text{step}])^\circ$, $! : \text{Digit} \times \text{Interval} \rightarrow 1$, $Q \triangleq (l^\circ !^\circ r) \cup 1$ l, r are coprod-laws's injections into $\text{FX} = 1 + (\text{Digit} \times X)$, so $l : 1 \rightarrow \text{FX}$ and $r : \text{Digit} \times X \rightarrow \text{FX}$, $w \triangleq 2^{17}$.

 $([\text{arb}, \text{step}])^\circ \text{Eval inrange}^\circ$

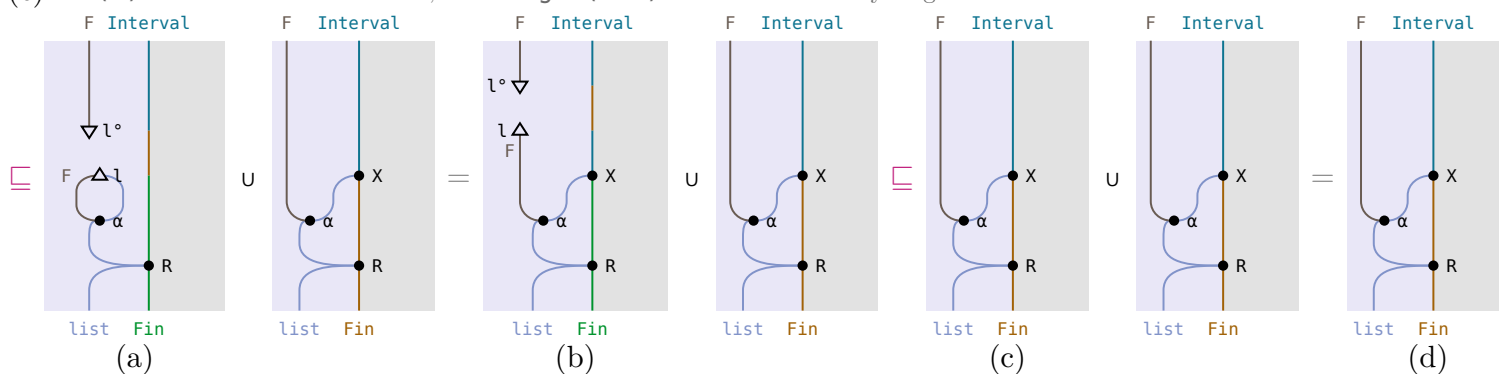
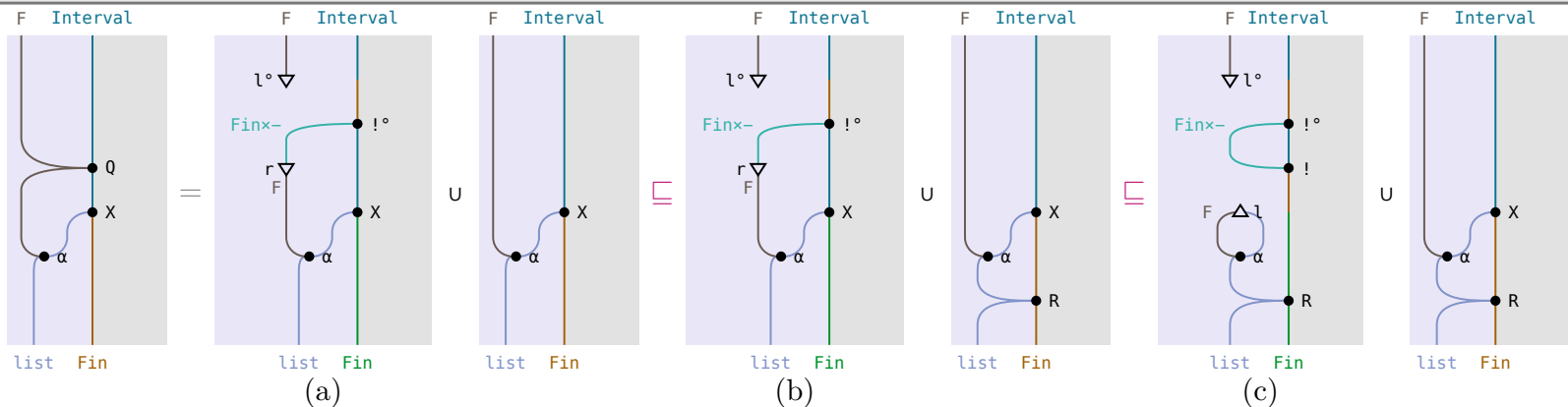
(10.4b)

every interval got by folding a decimal's digits with $[\text{arb}, \text{step}]$ has the decimal's value strictly inside it

 $QF(X) \alpha \sqsubseteq EF(X) \alpha R$

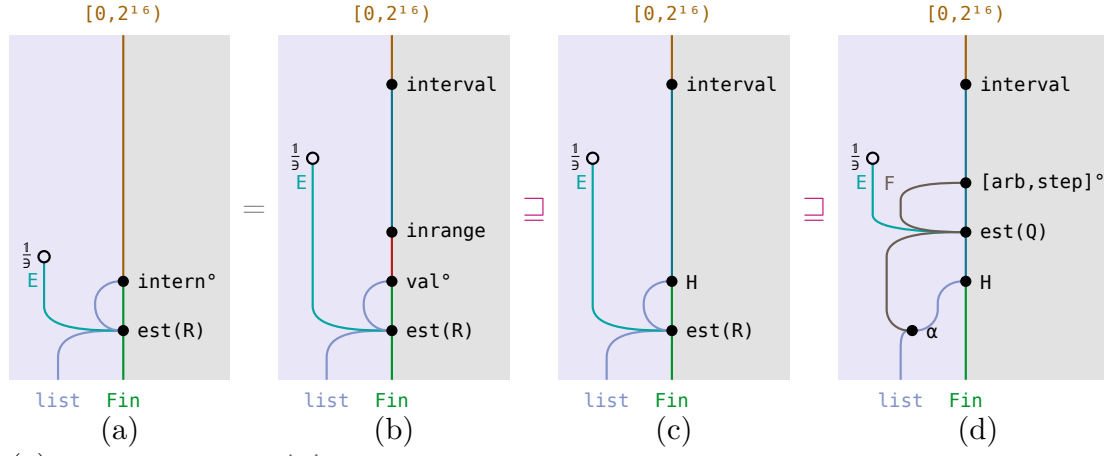
(10.4c)

choosing by Q among the one-step decompositions before building with $F(X)$ and α yields only decimals that are R -related to one built without choosing



$$\text{interval}(\mu X : \frac{[\text{arb}, \text{step}]^\circ}{3} \text{est}(Q)F(X)\alpha) \sqsubseteq \frac{\text{intern}^\circ}{3} \text{est}(R) , \quad \text{extern}(n)=f(2n-1, 2n+1) \quad (10.4d)$$

a shortest decimal whose internal representation is the given multiple of 2^{-16} is got by emitting the one digit the interval of admissible reals allows, until that interval contains zero and the empty decimal will do



- (a) the specification — $\triangleq$ **intern**
 (b) **round**[°] is not a map, but **interval** is, so it comes out of the transpose
 (c) the type restriction (10.9): a shortest decimal **H** gives an interval is a shortest one among all decimals inside it, and is inside it by (10.4b)
 (d) $\frac{[\text{arb}, \text{step}]^\circ}{3}$ returns at most two elements — stop, or take one more digit — and ! **nil**_{cons} **R**[°] makes it stop whenever stopping is legal

- $f(a,b)=x \iff a < 0 \wedge x=[] \vee \neg a < 0 \wedge (\exists d. \exists y. d = \lfloor 10b \rfloor \wedge f(10a-d, 10b-d)=y \wedge x=[d]\#y)$
 f gives (a,b) the empty decimal when $a < 0$, and otherwise the digit $d=\lfloor 10b \rfloor$ followed by what it gives $(10a-d, 10b-d)$ (10.4e)
- $(\mu X : \frac{[\text{arb}, \text{step}]^\circ}{3} \text{est}(Q)F(X)\alpha)=f$
 the least solution of the greedy recursion of (10.4d) is f
- $d = \lfloor 10b \rfloor \wedge e = \lfloor 10b \rfloor \implies d=e$
 Exercise 10.12: two digits d and e with $0 < 10b-d < 1$ and $0 < 10b-e < 1$ are equal
- $(0 < q-p \wedge q < w) \wedge p < 0 \implies f(p,q)=[]$
 the program's first clause: $f(p,q)=[]$ when $p < 0$
- $(0 < q-p \wedge q < w) \wedge \neg p < 0 \wedge d=10q/w \implies f(p,q)=[d]\#f(10p-wd, 10q-wd)$
 the program's second clause: with $d=(10q) \text{ div } w$, $f(p,q)$ is d followed by $f(10p-w \cdot d, 10q-w \cdot d)$
- $(0 < q-p \wedge q < w) \wedge \text{Reach}(p,q) \implies f(p/w, q/w)=f(p,q)$
 on the pairs the program reaches from **interval** n , the integer f at (p,q) is a value of the rational f at $(p/w, q/w)$
- **interval** $f=\text{extern}$
 f after **interval** is the program **extern**, whose value at n is $f(2n-1, 2n+1)$

§11. Appendix

§11.1. $P(S) \text{ est}(R) = (\exists S) n(\epsilon \setminus (SR^\circ))$

(11.1a)

(p, q) tabulates $W \triangleq (\exists S) n(\epsilon \setminus (SR^\circ))$, and $y \triangleq \frac{(p \exists S) n(qR)}{\exists}$, a map.

(11.1b)

$$\begin{array}{c} \boxed{\begin{array}{c} W \\ P(S) \text{ est}(R) \end{array}} \xleftarrow{p^\circ q = W, 1 \sqsubseteq yy^\circ} \begin{array}{c} \boxed{\begin{array}{c} p^\circ y \\ P(S) \end{array}} \\ \boxed{\begin{array}{c} y^\circ q \\ \text{est}(R) \end{array}} \end{array}$$

(11.1c)

$$\begin{array}{c} \boxed{\begin{array}{c} y^\circ q \\ \exists \end{array}} \xleftarrow{f^\circ \cdot \neg f \cdot, \cdot \exists \neg \exists} \boxed{\begin{array}{c} q \\ (p \exists S) n(qR) \end{array}} \xleftarrow{\Delta \neg n, f^\circ \cdot \neg f \cdot} \begin{array}{c} \boxed{\begin{array}{c} p^\circ q \\ \exists S \end{array}} \\ \boxed{\begin{array}{c} 1 \\ R \end{array}} \end{array}$$

R reflexive

(11.1d)

$$\begin{array}{c} \boxed{\begin{array}{c} \epsilon y^\circ q \\ R^\circ \end{array}} \xleftarrow{\cdot \exists \neg \exists, \circ, \text{meet}} \boxed{\begin{array}{c} R^\circ q^\circ q \\ R^\circ \end{array}} \quad \text{q simple} \\ \\ \begin{array}{c} \boxed{\begin{array}{c} p^\circ y \\ P(S) \end{array}} \xleftarrow{\Delta \neg n, \cdot T \neg / T, \circ, f^\circ \cdot \neg f \cdot} \begin{array}{c} \boxed{\begin{array}{c} p^\circ y \exists \\ \exists S \end{array}} \\ \boxed{\begin{array}{c} p \exists \\ y \exists S^\circ \end{array}} \end{array} \\ \\ \begin{array}{c} \boxed{\begin{array}{c} p^\circ y \exists \\ \exists S \end{array}} \xleftarrow{\cdot \exists \neg \exists, \text{meet}} \boxed{\begin{array}{c} p^\circ p \exists S \\ \exists S \end{array}} \quad \text{p simple} \\ \\ \boxed{\begin{array}{c} p \exists \\ y \exists S^\circ \end{array}} \xleftarrow{\text{modular law}} \boxed{\begin{array}{c} p \exists \\ (p \exists) n(qRS^\circ) \end{array}} \xleftarrow{1 \sqsubseteq qq^\circ} \boxed{\begin{array}{c} q^\circ p \exists \\ RS^\circ \end{array}} \xleftarrow{\circ, T \cdot \neg T \setminus} \boxed{\begin{array}{c} W \\ \epsilon \setminus (SR^\circ) \end{array}} \quad \text{W's right half} \end{array}$$

§11.2. $P(\text{est}(R)) \text{ est}(R) = P(\text{dom}(\text{est}(R))) \text{ union est}(R)$

(11.2a)

$$\begin{array}{c} \boxed{P(\text{est}(R)) \text{ est}(R)} \xrightarrow{\text{dom}(\text{est}(R)) \text{ est}(R) = \text{est}(R)} \boxed{P(\text{dom}(\text{est}(R)) \text{ est}(R)) \text{ est}(R)} \\ \\ \boxed{P(\text{dom}(\text{est}(R))) P(\text{est}(R)) \text{ est}(R)} \xrightarrow{\text{est-711}} \boxed{P(\text{dom}(\text{est}(R))) \text{ union est}(R)} \end{array}$$

P a relator

(11.2b)

$$\begin{array}{c} \boxed{\begin{array}{c} P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ P(\text{est}(R)) \text{ est}(R) \end{array}} \xleftarrow{(11.1b) \text{ at } S := \text{est}(R), \Delta \neg n, T \cdot \neg T \setminus} \boxed{\begin{array}{c} P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ \exists \text{est}(R) \end{array}} \\ \\ \boxed{\begin{array}{c} \epsilon P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ \text{est}(R) R^\circ \end{array}} \xleftarrow{\text{count of } T \cdot \neg T \setminus} \boxed{\begin{array}{c} \exists (\exists n(\epsilon \setminus (\epsilon \setminus (SR^\circ)))) \\ \exists (\exists n(\epsilon \setminus (SR^\circ))) \end{array}} \\ \\ \boxed{\begin{array}{c} P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ \exists \text{est}(R) \end{array}} \xleftarrow{P(\text{dom}(\text{est}(R))) \sqsubseteq 1} \boxed{\begin{array}{c} \text{union est}(R) \\ \exists \text{est}(R) \end{array}} \\ \\ \boxed{\begin{array}{c} \epsilon P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ \text{est}(R) R^\circ \end{array}} \xleftarrow{\epsilon \text{ lax natural}} \boxed{\begin{array}{c} \text{dom}(\text{est}(R)) \epsilon \text{ union est}(R) \\ \text{est}(R) R^\circ \end{array}} \\ \\ \boxed{\begin{array}{c} \text{est}(R) \text{ est}(R)^\circ \epsilon \text{ union est}(R) \\ \text{est}(R) R^\circ \end{array}} \xleftarrow{\text{dom}(\text{est}(R)) \sqsubseteq \text{est}(R) \text{ est}(R)^\circ} \boxed{\begin{array}{c} \text{est}(R) \epsilon \text{est}(R) \\ \text{est}(R) R^\circ \end{array}} \quad \text{UP of est} \end{array}$$

§11.3. Shortest paths on a cylinder, on lists

(11.3a)

definition	type	note
$F(A, X) = A + A \times X$	$\mathcal{A} \rightarrow \mathcal{A}$	The base functor: one square of the new column, alone or in front of the path so far.
$L = \text{list}^+$	$\mathcal{A} \rightarrow \mathcal{A}$	A path is a non-empty list of squares, one per column crossed.
α the initial algebra	$F(\text{dCL}(L, E)) \rightarrow \text{dCL}(L, E)$	$\alpha(5) = [5]$ and $\alpha(1, [5]) = [1, 5]$: a path is started by one square, or extended by one.
N the n-tuple relator	$\mathcal{A} \rightarrow \mathcal{A}$	One component per row of a column, so the fold carries n answers at once.
$R \triangleq \text{sum} \leq \text{sum}^\circ$	$[\text{Nat}] \rightarrow [\text{Nat}]$	The cost of a path, which the cheapest minimises.
setify	$A[n] \rightarrow EA$	$\text{setify}(1, 2, 3, 4) = \{1, 2, 3, 4\}$ — which row a component came from is forgotten.
moves	$A[n] \rightarrow E(A[n])$	$\text{moves}(x) = \{\text{up}(x), x, \text{down}(x)\}$ — rotated up, unrotated, rotated down.
trans	$E(A[n]) \rightarrow (E A)[n]$	$\text{trans}\{(a, b, c), (x, y, z)\} = (\{a, x\}, \{b, y\}, \{c, z\})$ — component k of the result is the set of the k-th components.
zip	$F(a[n]) \rightarrow (F a)[n]$	$\text{zip}((1, 2, 3, 4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})) = ((1, \{[5]\}), (2, \{[6]\}), (3, \{[7]\}), (4, \{[8]\}))$.
$\text{cp} \triangleq \frac{F(1, \exists)}{\exists}$	$F(EA) \rightarrow EFA$	$\text{cp}(1, \{[5], [6], [8]\}) = \{(1, [5]), (1, [6]), (1, [8])\}$.
$\text{gen} \triangleq F(1, \text{moves trans } N(\text{union})) \text{zip } N(\text{cp } E(\alpha))$	$F(NA, N(E(TA))) \rightarrow N(E(TA))$	$\text{gen}((1, 2, 3, 4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\}))$ is worked out in (11.3.1b).
$\text{paths} \triangleq (\text{gen}) \text{setify union}$	$T(NA) \rightarrow E(TA)$	$\text{paths}[(1, 2, 3, 4), (5, 6, 7, 8)]$ is the union of (11.3.1b)'s four sets: 12 paths, 3 from each entry row.
the specification $\text{paths est}(R)$	$[\text{Nat}[n]] \rightarrow [\text{Nat}]$	A cheapest path from the entry side to the exit side.

§11.3.1. $N(E(L A))$

$$\begin{array}{l} 1 \quad 5 \\ 2 \quad 6 \\ 3 \quad 7 \\ 4 \quad 8 \end{array} \quad [(1, 2, 3, 4), (5, 6, 7, 8)] : L \ N \ \text{Nat}$$

(11.3.1a)

$(\text{gen})[(5, 6, 7, 8)] = (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})$

(11.3.1b)

$$\begin{aligned} \text{gen}((1, 2, 3, 4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})) = \\ (\{ [1, 5], [1, 6], [1, 8] \}, \\ \{ [2, 5], [2, 6], [2, 7] \}, \\ \{ [3, 6], [3, 7], [3, 8] \}, \\ \{ [4, 5], [4, 7], [4, 8] \}) \end{aligned}$$

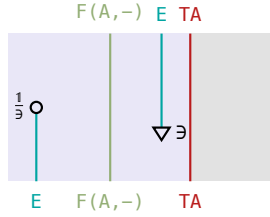
(11.3.1c)

N	the 4-tuple, one component per row of the entry column
E	the set of paths that can start in that row
L	a path is the non-empty list of the squares it crosses

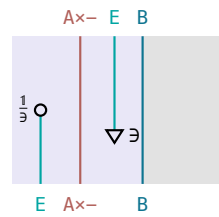
row 1's set has no $[1,7]$ — from row 1 only rows 4, 1 and 2 are reachable — and it does have $[1,8]$, because the cylinder glues the bottom row to the top. That is **moves trans N(union)** of $\triangleq$ **paths**: component k collects the paths of rows $k-1, k, k+1$.

§11.3.2. The cross product $\text{cp} \triangleq \frac{F(1, \exists)}{\exists}$

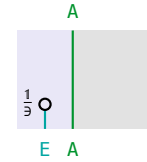
(11.3.2a)



cp on all of $F(A, E(L A))$



the $A \times -$ summand: $\exists$ picks one path, $1\% \exists$ collects the results



the A summand, no E to distribute: $1\% \exists$ alone, $a \mapsto \{a\}$

(11.3.2b)

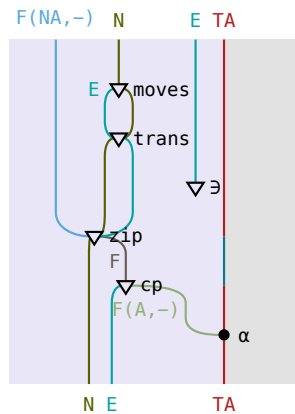
type	note
$F(A, -) = A + A \times -$	one wire carries the whole functor, both summands with it; the $\exists$ under it is $F(1, \exists)$
$\exists : E(L A) \rightarrow L A$	the $\exists$ inside $F(1, \exists)$: a set of paths, one of them — $L A$ is one path, A one square
$F(1, \exists) = 1 + 1 \times \exists$	$: A + A \times E(L A) \rightarrow A + A \times L A$, the relator acting on each summand
$\exists : E(A + A \times L A) \rightarrow A + A \times L A$	the $\exists$ under the bar: a set of cells, one of them — a different $\exists$
$\text{cp} : A + A \times E(L A) \rightarrow E(A + A \times L A)$	$\frac{R}{\exists}$ turns $R: X \rightarrow Y$ into $X \rightarrow E Y$
$\text{cp} = \frac{1}{\exists} E(F(1, \exists))$	what the picture draws: the unit makes the outer E , leaving one $\exists$ — pow-laws

(11.3.2c)

$w = (1, \{[5], [6], [8]\}) : A \times E(L A)$ the right summand
 $1 \times \exists : A \times E(L A) \rightarrow A \times L A$ 1 keeps the square 1 , $\exists$ picks one path
 $w \mapsto (1, [5]), (1, [6]), (1, [8]) : A \times L A$ one output per path in w
 $\text{cp}(w) = \{(1, [5]), (1, [6]), (1, [8])\} : E(A + A \times L A)$ the three of them, collected

§11.3.3. $\text{gen} = F(1, \text{moves trans N(union)}) \text{ zip N(cp P}(\alpha))$

(11.3.3a)



(11.3.3b)

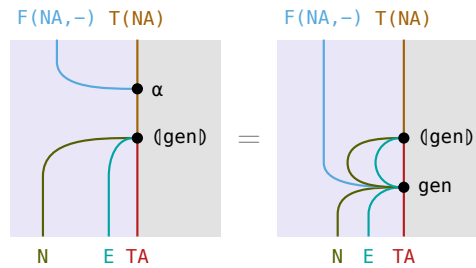
```

u = ((1,2,3,4),({[5]},{[6]},{[7]},{[8]})) : F(N A,N(E(L A)))
F(1,moves trans N(union))                1 keeps the column, the path SETS move
  moves({[5]},{[6]},{[7]},{[8]})
    = ({[6]},{[7]},{[8]},{[5]}),
      ({[5]},{[6]},{[7]},{[8]}),
      ({[8]},{[5]},{[6]},{[7]})}          : E(N(E(L A)))  down, unmoved, up
  trans(that)
    = ({[6]},{[5]},{[8]},{[7]},{[6]},{[5]}},
      {[8]},{[7]},{[6]},{[5]},{[8]},{[7]}) : N(E(E(L A)))  row k gets rows k-1, k, k+1
  N(union)(that)
    = ({[5],[6],[8]},{[5],[6],[7]},
      {[6],[7],[8]},{[5],[7],[8]})        : N(E(L A))      every path into row k, none dropped
zip(that)
  = ((1,{[5],[6],[8]}),(2,{[5],[6],[7]}),
    (3,{[6],[7],[8]}),(4,{[5],[7],[8]})) : N(F(A,E(L A)))  each row: its square, and the
                                                                paths it may be put in front of
N(cp P(α))(that)
  = ({[1,5],[1,6],[1,8]},{[2,5],[2,6],[2,7]},
    {[3,6],[3,7],[3,8]},{[4,5],[4,7],[4,8]}) : N(E(L A))    cp pairs the square with each path,
                                                                α prefixes it: α(1,[5])=[1,5]

```

§11.3.4. gen is an F-algebra; (gen): α(gen)=F(1,(gen))gen

(11.3.4a)



α puts the column back on the list, then the fold reads all of it

the fold reads the rest under F, then one gen puts the column in front

$$xs = [(1,2,3,4),(5,6,7,8)] = \alpha((1,2,3,4),[(5,6,7,8)]) : L(N A)$$

(11.3.4b)

$$(\text{gen})(xs) = \text{gen}((1,2,3,4), (\text{gen})[(5,6,7,8)])$$

both halves are in (11.3.1b): the tail folds to $(\{[5]\},\{[6]\},\{[7]\},\{[8]\})$, one path per row and each of them one square long, and gen on it is (11.3.3b)'s walk.

§11.3.5. Q : F(N A,N(L A))→N(L A), an F(N A,-)-algebra: Q=F(1,moves trans N(est(R))) zip N(α)

(11.3.5a)

```

Q : F(N A,N(L A)) → N(L A)                the fold's algebra: a new column, one path per row
u = ((1,2,3,4),([5],[6],[7],[8]))          : F(N A,N(L A))
F(1,moves trans N(est(R)))                  1 keeps the column, the paths move
  moves([5],[6],[7],[8])
    = ({[6],[7],[8],[5]},
      {[5],[6],[7],[8]},
      {[8],[5],[6],[7]})}                  : E(N(L A))      down, unmoved, up
  trans(that)
    = ({[6],[5],[8]},{[7],[6],[5]},
      {[8],[7],[6]},{[5],[8],[7]})        : N(E(L A))      row k gets rows k-1, k, k+1
  N(est(R))(that)
    = ([5],[5],[6],[5])                    : N(L A)          the cheapest into each row –
                                                                chosen BEFORE the new square
zip(that)
  = ((1,[5]),(2,[5]),(3,[6]),(4,[5]))      : N(F(A,L A))    each row: its square, and the one
                                                                predecessor that survived
N(α)(that)
  = ([1,5],[2,5],[3,6],[4,5])              : N(L A)          α(1,[5])=[1,5]

```

$Q = [N(\text{wrap}), (1 \times \text{moves trans } N(\text{est}(R))) \text{ zip}' N(\text{cons})]$ (11.3.5b)	
at the last column Q starts one path per row, and at each earlier one it puts each square in front of the cheapest of the three kept paths it can step to — the algebra (11.3.5c)'s last step folds	
circuit — the fork is $F(NA, N(LA)) = NA + NA \times N(LA)$	Hinze–Marsden
$F(N A) (N (T A)) \text{---} \boxed{Q} \text{---} N(T A)$	
$= FN(T A) \text{---} \boxed{F(1, \text{moves trans } N(\text{est}(R)))} \text{---} FN(T A) \text{---} \boxed{\text{zip}} \text{---} N(F A (T A)) \text{---} \boxed{N(\alpha)} \text{---} N(T A)$ the fusion condition (11.3.5d) read as a definition,	
$= F(N A) (N (T A)) \text{---} \left[\begin{array}{l} N(p_{\circ}) \text{---} \boxed{N(\text{wrapA})} \\ q \text{---} \boxed{\text{pm}} \text{---} q \text{---} \boxed{\text{zip}'} \text{---} N(q_{\circ}) \text{---} \boxed{N(\text{consA})} \end{array} \right] \text{---} N(T A)$ $\text{zip} = 1 + \text{zip}', \alpha = [\text{wrap}, \text{cons}]$	

paths $\text{est}(R) \exists (Q)$ setify $\text{est}(R)$ (11.3.5c)	
shortest paths on a cylinder: the dynamic programming approach to exhaustive search allows a path of least cost to be found in $O(n \times m)$ time; Q is (11.3.5b)'s algebra.	
circuit	Hinze–Marsden
$T(N\ A) \text{---} \boxed{\text{paths}} \text{---} E(T\ A) \text{---} \boxed{\text{est}(R)} \text{---} T\ A$	
$= T(N\ A) \text{---} \boxed{F(N\ A)\ (N\ (E\ (T\ A)))} \text{---} \boxed{\text{gen}} \text{---} NE(T\ A) \text{---} \boxed{\text{setify}} \text{---} EE(T\ A) \text{---} \boxed{\text{union}} \text{---} E(T\ A) \text{---} \boxed{\text{est}(R)} \text{---} T\ A$ $\triangleq \text{paths at paths}$	
$\sqsubseteq T(N\ A) \text{---} \boxed{F(N\ A)\ (N\ (E\ (T\ A)))} \text{---} \boxed{\text{gen}} \text{---} NE(T\ A) \text{---} \boxed{\text{setify}} \text{---} EE(T\ A) \text{---} \boxed{P(\text{est}(R))} \text{---} E(T\ A) \text{---} \boxed{\text{est}(R)} \text{---} T\ A$ $P(\text{est}(R))\ \text{est}(R) \sqsubseteq \text{union}\ \text{est}(R) \text{ --- a minimum in each set, then a minimum of those; } R \text{ transitive}$	
$\sqsubseteq T(N\ A) \text{---} \boxed{F(N\ A)\ (N\ (E\ (T\ A)))} \text{---} \boxed{\text{gen}} \text{---} NE(T\ A) \text{---} \boxed{N(\text{est}(R))} \text{---} N(T\ A) \text{---} \boxed{\text{setify}} \text{---} E(T\ A) \text{---} \boxed{\text{est}(R)} \text{---} T\ A$ $\text{setify lax natural}$	
$\sqsubseteq T(N\ A) \text{---} \boxed{F(N\ A)\ (N\ (T\ A))} \text{---} \boxed{Q} \text{---} N(T\ A) \text{---} \boxed{\text{setify}} \text{---} E(T\ A) \text{---} \boxed{\text{est}(R)} \text{---} T\ A$ $\text{fusion at (11.3.5d)}$	

gen $N(\text{est}(R)) \exists F(1, N(\text{est}(R))) Q$ (11.3.5d)	
fusion: the condition for fusion in (11.3.5c)'s last step, used to derive a definition of Q .	
circuit	Hinze–Marsden
$F(N\ A)\ (N\ (E\ (T\ A))) \text{---} \boxed{\text{gen}} \text{---} NE(T\ A) \text{---} \boxed{N(\text{est}(R))} \text{---} N(T\ A)$	
$\sqsubseteq FNE(T\ A) \text{---} \boxed{F(1, N(\text{est}(R)))} \text{---} FN(T\ A) \text{---} \boxed{Q} \text{---} N(T\ A)$ $(7.13), \text{ then zip, trans, moves lax natural}$	

(7.13) is $F(1, \text{est}(R)) \alpha \sqsubseteq \text{cp } P(\alpha) \text{ est}(R)$, mon-thm71 at the map α with $\frac{F(1, \exists) \alpha}{\exists} = \text{cp } P(\alpha)$: extending every path in a set and then taking a minimum is beaten by extending one minimum. It is the crux here, not the greedy theorem.